Reflection-based heterogeneous migration of computations *

Anolan Milanés[1], Noemi Rodriguez[2], Roberto Ierusalimschy[2]

[1]Department of Computer Science
Federal Center for Technological Education of Minas Gerais (CEFET-MG)
Av. Amazonas, 7675 - Nova Gameleira – Belo Horizonte - Minas Gerais – Brazil

[2]Department of Computer Science - Pontifical Catholic University of Rio de Janeiro
Rua Marquês de São Vicente, 225, Gávea – 22453-900 – Rio de Janeiro – Brazil

anolan@decom.cefetmg.br, {noemi,roberto}@inf.puc-rio.br

Abstract. *Implementing heterogeneous migration of computations is hard: it demands knowledge of the type of the data in order to be able to capture and restore the computational state. Support for those operations has traditionally been offered through ready-made solutions for specific applications, which are difficult to tailor or adapt to different needs. A more promising approach would be to build specific solutions as needed, over a more general capture and restoration framework. That flexibility can be achieved through computational reflection. This work extends the Lua programming language with a reflective API that provides the programmer fine control over the capture and restoration mechanisms.*

1. Introduction

Different migration applications may have different requirements. Migration can be applied in *Opportunistic computing*, when a process that was created in a machine may need to be transferred to another host if the local user reclaims the machine. This may require the computation to be suspended, probably from outside the program, captured with all its dependencies and open files and transmitted and restored to another host, where part of the required libraries could be already in place. Similar requirements may be posed in load balancing, when the computation is moved to another, less loaded host, in order to improve its performance. On the other hand, a mobile agent may require to take along only part of its state, since at destination it will be bound to the new local context.

Systems providing support for migration, persistence, monitoring, or debugging, all have in common the need of dealing with the non-trivial problem of capturing and restoring the execution state of a computation. It would thus be natural to expect a fair amount of reuse or sharing among the developers of such systems. However, decisions regarding issues such as what should be captured, at what moment, and what strategy to choose for exception handling, vary a lot depending on the intended application. Traditionally, decisions about these issues are ingrained in systems with support for migration or persistence instead of providing common support for the general case [Milanés et al. 2008], hampering the possibilities of reusing them for purposes even slightly different from the original ones.

*This work has been partially supported by CNPq Brasil

Factoring out the support for capture and restoration from other system decisions in the form of general mechanisms avoids the need for building new systems from scratch every time a new requirement for state capture and restoration arises. Those mechanisms can then be used to build high-level libraries that implement specific policies for different application areas.

In our search for basic mechanisms for state capture and restoration, it is natural to turn to reflection: the ability of a programming system to observe and change its own behavior through reification and installation. That requires implementing mechanisms for the reification and installation of the execution state. *Reification* mechanisms allow the execution state information to be made available to the programmer as first-class values. *Installation* (also called *Reflection* [Friedman and Wand 1984]) allows the programmer to modify the program state by incorporating data from the program into the execution state. Using reflection, migrating a running computation would be reduced to reifying the continuation of a suspended execution, and then installing this continuation where desired. Similarly, persisting a running computation would involve reifying the continuation of the suspended execution, and installing it on the same host sometime in the future.

However, not any reification and installation will suffice for our needs. The programmer needs control over what parts of the computation will be captured, restored, and possibly bound with other, existing values. Simply starting a capture and letting it go without control may result in retrieving huge amounts of information, which may not be necessary if available at destination, for instance in load balancing. Furthermore, in order to make migration or persistence feasible when the value contains non-portable data, the programmer may need to manipulate the reified representation before it is re-installed. Reified objects must thus expose a structure that the programmer can manipulate in the form of fine-grained first-class values. If the programmer can handle such fine-grained values, (s)he should be able to easily de-compose and inspect them to discover nested references to other values and decide whether these should or not be reified. Navigating down the reified values, (s)he can create arbitrary data structures and use all available language features to code decisions about granularity, error-handling, and other issues.

Note that this approach is different from treating computations as first-class values, as in higher order languages [Clinger et al. 1999]. Those languages can treat computations as black-boxes. Our motivation for manipulating continuations is not their use as control structures, but their importance for applications such as migration and persistence. In our case, the program must be capable to inspect the internals of computations, which must thus be "reifiable" data structures.

In this paper, we experiment the proposed approach with *LuaNua*, an extension to the Lua programming language [Ierusalimschy et al. 2007] with mechanisms for the fine-grained *reification* and *installation* of Lua values. Heterogeneity in that case refers to the possibility of transferring the state of a running Lua program across different architectures and/or operating systems.

The rest of this paper is organized as follows. Section 2 provides an overview of the state of the art on flexible support for heterogeneous capture and restoration of the execution state of running computations. Section 3 describes our approach for the design of state manipulation mechanisms, proposes an API for reifying and installing Lua

values and explores its functionality through some examples. Section 4 describes two benchmark applications we have developed to analyze the impact of the proposed API on performance. Conclusions are given in section 5.

2. Related Work

As regards the state capture and restoration in Lua, the Pluto library [Sunshine-Hill 2008] already offers that support. However, in Pluto the captured values are not reified into the language, but directly translated into bytestrings. Among other consequences, this means that it is not possible to restore sharing correctly if values are captured independently.

The most distinguishing feature in LuaNua is the ability to capture and manipulate, as program data, parts of the execution stack. This ability relies heavily on Lua *coroutines* [Moura and Ierusalimschy 2009]. It is not common for languages to provide coroutines, but many provide support for continuations, which are in many ways similar to coroutines. Languages that provide support for capturing continuations rely basically on three methods: (i) the continuations can be captured transparently and dumped, (ii) the programmer may provide custom capture and restoration procedures, and (iii) the programmer may insert marks to indicate limits for marshaling.

An example of the first approach is Smalltalk. Smalltalk's stack inspection provides first-class access to the current execution stack via `thisContext` as a chain of linked stack frames called contexts [Rivard 1996]. The contexts can be captured as complete snapshots and persisted as part of the image, in the form of dumps. The idea is to persist the complete working environment, allowing that exact point of execution to be reinstated, and not, as in our case, to provide generic support for persistence and mobility.

Indeed, capturing a continuation may be ineffective, unnecessary and even unfeasible, when it comes to data that are not portable, such as file descriptors or channels. Stackless Python (version 2.6) provides the ability to capture and restore tasklets (lightweight threads) in a platform-independent manner. However, it is possible that a tasklet can not be restored, for instance, if its dump contains calls to C functions [Stackless 2011]. In this direction, some systems/languages, such as Gambit-C and Pluto, allow the programmer to specify custom capture and restoration procedures in an instance or type basis [Germain et al. 2006, Sunshine-Hill 2008]. In contrast, the stepwise character of our proposal allows for better control over the extension and content of the capture.

Perhaps the approach that is most closely related to ours is that of *delimited continuations*, which allow the programmer to specify, in the code, what part of the remaining execution should be captured in a continuation. Scala continuations [Rompf et al. 2009] are created by means of a program transformation into CPS of the code annotated with `shift-reset` keywords. Those marks allow the programmer to limit the extent of the capture, and enhance the portability of Scala continuations. This approach has been followed also in Swarm [Clarke 2013] for the migration of computations in a distributed system.

3. Proposal

This section presents the main design features of the current proposal. We describe our approach for the fine-grained support for capture and restoration, present a brief description of Lua, describe the proposed API, and finally discuss some examples of use.

3.1. A fine-grained reflective approach for capture and restoration

We have argued for handling the representation of the execution state as fine-grained first-class values. To that end, we need:

- *reification* mechanisms to allow the programmer to obtain these first-class representations of the program state in the form of data structures that can be freely navigated and manipulated.
- symmetric *installation* mechanisms for installing these program-level representations as part of the program state.

Those are the main operations provided by the API we propose. A reification following

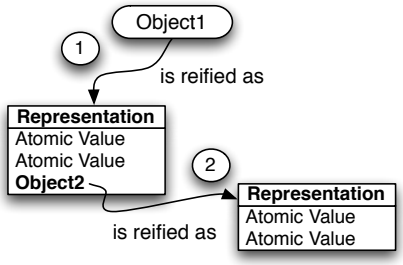

Figura 1. Reification step by step

this approach is executed top-down and step-by-step. Capturing an object requires the recursive reification of all the references composing its transitive closure, but the programmer can decide at every step whether to continue inspecting them all. The representation may consist of atomic values, such as numbers, and of complex values, such as functions or activation records. Figure 1 illustrates a step-by-step reification. The reification of *Object1* retrieves its internal representation, which includes two atomic values (that cannot be further reified) and the complex value *Object2*. To inspect *Object2*, the programmer must now reify it. In (2) we see the result of this reification: a representation that contains only atomic values. Here reification can continue no longer. A representation can be arbitrarily manipulated in order to satisfy application requirements (for instance, reducing cost, or replacing non portable data). Installation follows a bottom-up approach, in the reverse direction we executed the reification. That is, in Figure 1 we begin from (2) until we get to (1). Thus, more internal components are installed first, then composed into the representation of the next value to be installed.

Basically, a reified representation is a data structure that contains a copy of the internal representation of the reified entity. Changes to either the reified representation or to the execution state itself do not affect each other until an explicit install operation is issued. This is because we expect these operations to be seldom executed. However, at the time the reification or the installation is issued, the correspondence holds.

We extended the Lua programming language [Ierusalimschy et al. 2007] with support for fine-grained reification and installation. We implemented primitives for *reification* of values (capture) and for their *installation* (restoration) as an extension of Lua 5.1 that we have called *LuaNua* (Portuguese for "Naked Lua"). Our API is clearly separable from the underlying base language, allowing applications that do not use the reflection/meta-programming API to be deployed independently of it.

3.2. A brief intro to Lua

Lua is an interpreted, procedural and dynamically-typed language, featuring garbage collection and lexical scoping. Lua values can be of type *nil*, *boolean*, *number*, *string*, *table*, *function*, *thread*, and *userdata*. Tables are the language's single data structuring mechanism and implement associative arrays, indexed by any value of the language except *nil*. *Metatables* allow programmers to change the behavior of tables, providing, for instance, a standard way to implement object orientation, through prototyping.

The type *thread* is used for Lua coroutines. Coroutines are lines of execution with their own stack and instruction pointer, sharing global data with other coroutines [Moura and Ierusalimschy 2009]. Unlike traditional threads (for instance, Posix threads), coroutines are collaborative: a running coroutine suspends execution only when it explicitly requests to do so. Closures and coroutines are first-class values in Lua.

Internally, Lua manipulates two additional types hidden from the language level: *proto* and *upval*. Values of type *proto* are function prototypes. Upvalues are non-local values, that is, values from enclosing scopes available to nested functions.

Lua has a number of reflective facilities which already provide partial support for reification and installation. In Lua, source code and bytecode chunks can be loaded and executed dynamically. Other reflective features include access to the environment and to the names of local variables. The simplicity of concurrency in Lua avoids the need to address problems of synchronization. Lua coroutines are *stackful*, meaning they can suspend (and restart) execution at an arbitrary level of nested function calls. Having coroutines as first class values allows a homogeneous treatment of data and computations.

Lua supports the reification of functions as strings of bytes. However, this representation is not easily handled, and may require translation in case the architectures are different. The language does not allow for the serialization of coroutines.

3.3. LuaNua API

The most important functions added by LuaNua to Lua are *reify* and *install*:

- *reify(value, [level])* receives a value as parameter, and returns the representation of its structure. This primitive accepts an optional second parameter, which makes sense only in the case of reification of coroutines and represents the level of the desired activation record [LuaNua 2013].
- *install(representation, type | value,[level])* receives two parameters: the representation and the type or the value to be rebuilt. If successful, an invocation of this function should return a value of the specified type. Like *reify*, *install* receives an additional parameter "level" for the installation of coroutines.

Functions `reify` and `install` are available from the debug namespace. For instance, to reify a closure func, we call

```
local t = debug.reify(func)
```

This maps the internal representation of the closure *func* to a Lua table and saves it in local variable *t*. Lua tables can be inspected in all their extension, thus this solution fulfills our requirement of navigability.

On the other hand, to install a closure we call:

```
local foo = debug.install(t, "function")
```

Representations returned by *reify* may contain atomic values (numbers, booleans, strings) and/or references to complex abstractions (tables, functions, upvalues, prototypes, threads, userdata). The reification of an atomic value returns the value itself. Complex values are reified as a Lua table with references to the internal components of the value, or the components itself if atomic. That representation can be navigated by the programmer who can then decide whether or not to reify inner components. In the specific case of coroutines, the grain of reification and installation is an activation record.

Installation requires a representation that contains all the components of the value to be installed. Thus, the internal components must be installed first, then the values they are components of, and so on, following a bottom-up approach in the reverse direction we followed on reification.

Reification of arbitrary extents of program state with LuaNua thus consists of the progressive construction of a representation of the execution, by traversing the tables returned by successive calls to *reify* and filling them in, as needed, with the results of new invocations. The resulting tables can then be serialized using standard language mechanisms when implementing migration or persistence.

In addition to creating new computations, the programmer should be able to modify existing ones. It may be the case that one wishes to replace a coroutine within the program instead of creating a new coroutine upon restoration. To support this, *install* receives, as an optional parameter, the coroutine where the installation will be made.

3.4. Exploring the API

In what follows, we present a series of examples to illustrate the flexibility afforded by LuaNua. The examples were executed saving the resulting state in a file and restoring the stored representation from this file in another instance of the Lua interpreter, thus emulating necessary steps for either migration or persistence.

3.4.1. Basics

First, we show how to reify and install a function using LuaNua. We choose a simple function (*inc*) that receives a single parameter and returns its value incremented by one.

```
local function inc(counter)
  return counter + 1
end
```

Using the LuaNua API, this function can be reified as follows (Comments in Lua begin with "--"; we are using the symbol --> for output.):

```
-- reify function inc
local tinc = debug.reify(inc)
print(tinc) --> {p = 0x532920}
```

The call to *debug.reify* returns a table containing the representation of function *inc*, that is, a reference to the function prototype and a reference to the lexical environment (for simplicity we do not discuss its reification here, as well as that of debug data contained in the representation). The reference to the function prototype is saved in field *p* of table *tinc* (*tinc.p*). Next, we proceed further into the representation by reifying the function prototype, so we make a further call to *debug.reify*:

```
local proto = debug.reify(tinc.p)
```

Now table *proto* contains the bytecode of the function prototype. Because all the values composing the representation are already atomic, reification ends here.

At this point, tables *proto* and *tinc* could be serialized and written to a file or transferred in a message. At some later point in time the tables could be reconstructed from the serialized representation and we would be ready to reinstall function *inc*.

For installation, we must follow from the bottom up. First, we install the internal, non-atomic values in memory space. In this case, the only such value is the function prototype, which was saved in table *proto*. Thus, we install the function prototype (and save it as tinc.p) and then re-create (install) the function represented by table tinc.

```
local tinc = {p = debug.install(proto, 'proto')}
local newinc = debug.install(tinc, 'function')
```

Now that newinc contains the installed function, we can execute it:

```
print(newinc(1)) --> 2
```

We can see in this example that the reification/installation manual procedure allows the programmer to control the composition of the representation. On the other hand, we see that types that were previously hidden (such as prototypes) are now visible. Because the language design does not assume that these values will be manipulated by the programmer, this visibility may lead to unanticipated execution errors (for instance, illegal operations on these values will not be handled elegantly).

3.4.2. Reification/installation of executions

A more interesting example is capturing and restoring executing computations. Lua provides asymmetric coroutines, which are controlled through calls to the *coroutine* module. A coroutine is defined through an invocation of *create* with an initial function as a parameter. The created coroutine can be (re)initiated by invoking *resume*, and executes until it invokes *yield*. For instance, function *count* is an iterator that, for each number from 1 to 5, prints the number and yields its value (In Lua, functions may return multiple values. *coroutine.resume* returns a value of *true* or *false* indicating whether the coroutine was resumed successfully and, optionally, values passed to *yield*):

```
-- definição da função
local function count()
   for i = 1, 5 do
      print("Number",i)
      -- send this number back to activator
      coroutine.yield(i)
   end
end
```

Suppose we want to execute *count* until it produces the number 3, and then capture the suspended coroutine. We can write the following code.

```
local coro = coroutine.create(count)
local status, i
repeat
   -- resume returns a status and
   -- the yielded values
   status, i = coroutine.resume(coro)
until (i == 3)
local rep = capture(coro)
```

When executed, this code prints:

```
--> Number  1
--> Number  2
--> Number  3
```

and stops.

Let's turn our attention to the implementation of *capture*. Internally, a Lua coroutine has a stack organized in activation records, each of them corresponding to an active function. We can reify a Lua coroutine by composing its reified frames.

We must iterate over the frames composing the stack, invoking *reify* for each level and unwinding every structured (that is, non atomic) value. This procedure creates a table with the representation of the requested entity and its components.

We begin by obtaining a reference *thr* to a new table:

```
local thr = {}
```

Then we save the coroutine status:

```
thr.status = coroutine.status(coro)
```

Now we iterate over the valid levels of the stack, from the top level:

```
repeat
  level = level + 1
  thr[level] = debug.reify(coro, level)
```

To complete the loop, we increase *level*, to move to the next activation record:

```
function capture(coro)
        local thr = {}
        repeat
          level = level + 1
          thr[level] = debug.reify(coro, level)
        until (thr[level]==nil)
        thr.status = coroutine.status(coro)
   return thr
end
```

Because a coroutine representation contains non-atomic data, its contents must in turn be reified. To that end, we inspect every activation record, searching for non atomic components. We then reify those values recursively until our representation is serializable. The activation record at level 0 corresponds to the call to `yield`. Because `yield` is a C function, it cannot be reified and serialized. The data that are neither portable nor available at the destination can be replaced by the corresponding pair during restoration.

The activation record at level 1 corresponds to the invocation of function `count`. Since that function is not part of the global environment, we need to take it along:

```
local func = thr[1].func --> count is at level 1
local t = debug.reify(func) --> t contains the representation of count
t.env = getfenv(func) --> capture the function environment
if t.env==_G then t.env = nil end --> we do not take along the global environment
t.isC = 0 --> not a C function
```

Function `count` has a prototype, that must also be reified:

```
local proto = debug.reify(t.p)
```

Now our representation is complete. At destination, in order to restore our computation, we need to install every value in the stack, and then install every activation record. We follow the reverse order: first we install the prototype, and then function count. After that, we are ready to install the execution stack. We create a new, empty coroutine, and then reconstruct the coroutine from the representation saved in *thr*.

```
local ncoro = debug.newthread()
for i = #thr, 0, -1 do
  ncoro = debug.install(thr[i], ncoro, 0)
end
debug.setstatus(ncoro,thr.status)
print(coroutine.status(ncoro)) --> suspended
```

Now the coroutine is ready for execution.

```
coroutine.resume(ncoro) —> Number 4
```

If only a partial execution is desired, it can be constructed from scratch with a subset of the reified activation records.

Although flexible, this approach is not scalable, and becomes infeasible for more complex computations. The next section describes the extension of an existing object-oriented library with higher-level functions based on the support provided by LuaNua.

3.5. Pickling library

LOOP (Lua Object-Oriented Programming) [Maia 2008] is a set of packages that allows the implementation of different object-oriented programming models in Lua and offers limited support for serialization. We have built migration and persistence libraries by extending the LOOP library with support for capturing and restoring Lua computations. Capture and restoration stages basically behave as follows.

State Capture We call the method `put` of the instance `stream` of the LOOP serializer with the suspended coroutine `co` as argument. As a result of this procedure a chunk of code, that allows re-creating the execution along with the environment, is stored and can then be serialized.

```
stream:put(co)
local msg = string.format(''%s'',table.concat(stream))
```

After this, the returned string can be easily persisted.

Capturing files introduces the problem of how to capture non-portable values. To solve it, we have redefined the `open` method of the IO library in order to register file access modes and the filepath when a file is opened. Only registered files will be captured, and their data must be stored at the serializer instance before serialization. At restoration, a function opens the file at the stored path with the access mode and position it had when captured, and returns the file handle.

State Restoration State can be restored from the serialized value (*buffer*) produced by the capture step. Similar to the capture, after loading the serializer libraries, we instantiate a serializer. Then we store the buffer in the serializer instance and call the deserialization method. The method returns the captured coroutine.

```
stream.data=buffer
local restoredCo = stream:get()
print(coroutine.status(restoredCo)) —> suspended
```

Now the platform can resume execution:

```
local status, error = coroutine.resume(restoredCo)
```

4. Experiments

To evaluate the cost of reification and installation procedures, we implemented capture and restoration of a program that calculates the factorial of a number, and migration of the n^{th} Fibonacci number and a k-NN (k-Nearest Neighbor) application. Those implementations use the pickling library described in Subsection 3.5. The experiments on capture and restoration were conducted to verify the behaviour of the checkpointing latency and the storage/load and capture/load delay ratios. Capture time is defined as the time of reflecting the computation into a string (since the extraction of the representation

of the computation as tables and its conversion to strings are performed together in the serialization functions, we did not separate those times). We measured the cost of migration (migration latency) compared to the execution time of the computation, to verify if this migration procedure is reasonable. The migration cost here is defined as the time between the beginning of the capture and the end of the load stage (after the computation is installed back). The execution time of computations refers to the time elapsed between the start of the execution and the return of the function (with the final result).

We also evaluated our proposal with a real application: a program executing the k-NN algorithm for the classification of a documents database.

The experiments were executed on two CPUs Intel Core 2 Duo 2.16 GHz machines with 1 GB de RAM connected to a 100Mb switch inside a local network. Both machines run GNU/Linux Fedora Core 8 kernel 2.6.25.4-10. Time is measured using the getrusage() function, and thus refers to the CPU time effectively used by the process. All time units are milliseconds. The code for all examples is available at [LuaNua 2013].

4.1. Capturing and restoring the Factorial recursive algorithm

Since capture and restoration times depend on the amount of transferred data, our first experiment measures the capture/storage and load/restoration times of a computation as the stack size increases. We execute the following implementation of the Factorial algorithm:

```
local function factorial(n)
    if n==0 then
        coroutine.yield()
        return 1
    else
        return n*factorial(n-1)
    end
end
```

and capture the coroutine in which the function is executing when a call to *yield* is issued.

We ran this code for values from 0 to 50 (with measurements at multiples of 5). Figure 2 plots the delay of capture and restore operations we obtained for different frame sizes. Because storage and load times are very similar and close to zero, we used a logarithmic scale on the **y** axis of Figure 3 to plot the time for capturing, saving, loading, and restoring operations for each frame size.

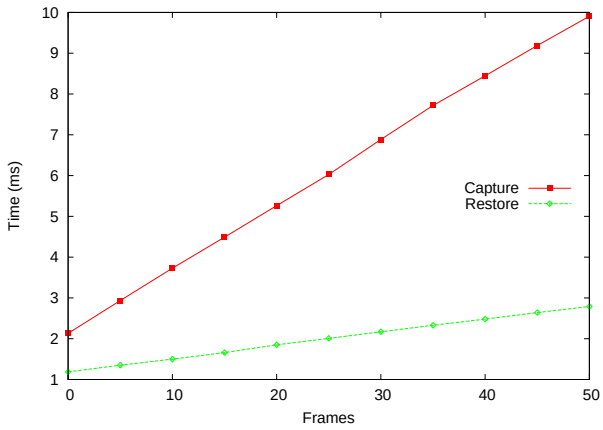

Figura 2. Capture/Restoration

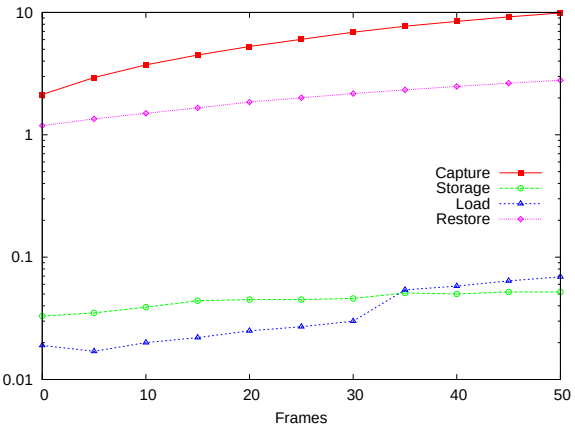

Figura 3. Capture/Store/Restoration/Load

The results show the higher complexity of capture when compared to restoration, which is due to the process of constructing the installation code that will be executed at restoration. It is also apparent that both capture and restoration times grow almost linearly with the number of frames on the stack, as expected, when the size of the data increases, while the delay for saving and loading keeps steadily negligible. That is, capture and restoration weigh far more than file-related procedures. Those results are rather different to other observations found in the literature([Bouchenak et al. 2004]). This is partly due to the verboseness of our library and also to the nature of the reification/installation procedure, which emphasizes flexibility over performance. Unlike Bouchenak's approach, we are mapping execution structures to language values and only then serializing the results.

4.2. Migration

In this section, we discuss an experiment with migration. We developed a simple migration platform, implemented two applications, and measured migration times. The first application is the Fibonacci recursive algorithm. For the second application, we chose the k-NN algorithm because it is a real application, is easy to implement in Lua, has a regular behaviour and forces us to consider aspects not usually present in simpler experiments, such as dealing with open files.

4.2.1. Migration Platform

Our migration platform is based on LuaNua, LOOP (see Section 3.5) and ALua [Ururahy et al. 2002], an event-based system for distributed programming. The programmer must explicitly insert suspension points across his code to allow migration to take place. When the application yields and the platform regains control, the application is captured and transferred. The captured computation is saved to a file that is copied to the destination, as are any open files. At the new host, the migration platform executes the received code to recreate the computation, which is then restarted.

To guarantee that all open files are transferred, we redefined function *io.open* to store information about open files in a table for later use.

The experiments were performed using two machines but, to avoid dealing with clock differences, the first machine, A, executes a computation, suspends it, captures its state in a file, sends it through the network to B (along with the open files, it also sends a message for B to initiate the operation of sending them back), from where the files are resent to A and then restored. *Migration time* was computed as the sum of capture, store, restoration and load time, plus the transmission time divided 2. The *Total execution time* is the time elapsed between the initiation of the coroutine and its return, including the time for migrating the coroutine from A to B.

4.2.2. Migration of a Fibonacci execution

We executed the following implementation of the Fibonacci recursive algorithm:

```
local stop = true
local function fibonacci(n)
   if n==0 then
       if stop then coroutine.yield(); stop = false; end
```

```
        return 0
    elseif n==1 then
        return 1
    else
        return fibonacci(n-1)+fibonacci(n-2)
    end
end
```

We included a suspension call that makes the program stop when the value of the parameter is 0. After the first suspension, a flag is set in order to allow the program to continue execution until the end. We have done this in order to measure the total execution time in presence of migration. (We could have also replaced function *yield* for a dummy function on the representation before reinstalling it, with a similar effect). The function was executed varying the Fibonacci's parameter value from 0 to 50 at intervals of 5 units. The results are shown in Figure 4. As the figure shows, the cost of the migration in relation to

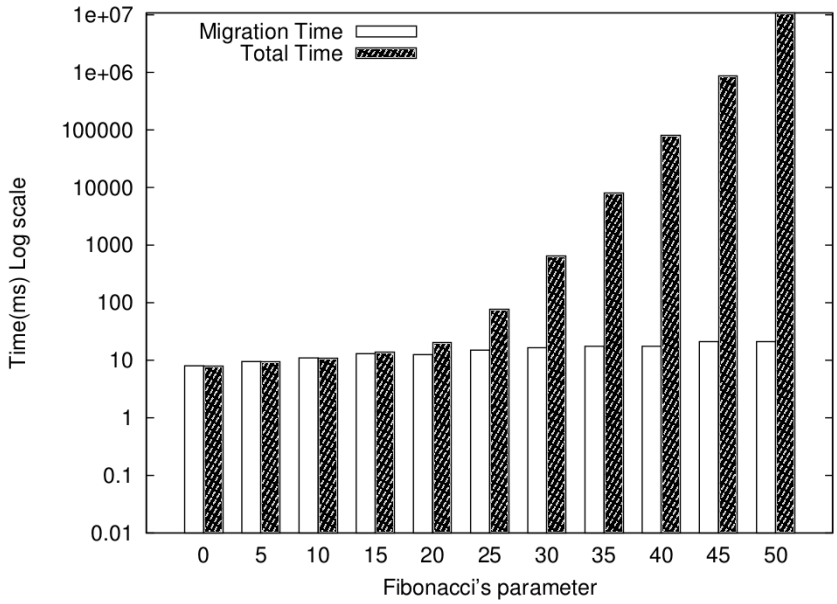

Figura 4. Migration of a Fibonacci execution

total execution time rapidly decreases with the increase of computation until it is almost negligible. This indicates that functions involving a certain amount of computation can benefit from migration for *Opportunistic Computing* or load balancing purposes, since the migration time is compensated by the computation time after a certain point. We can also see that migration time grows steadily but very slowly. This is because every activation record has a reference to the same function (the Fibonacci function), which is reified only once, and thus the amount of new information with every new activation record remains constant and small.

4.2.3. K-nearest neighbors algorithm (k-NN)

The k-nearest neighbors algorithm (k-NN) is a method for classifying objects based on the closest examples in a training set, frequently used in data mining applications. An

object is included in the most common class amongst its k-nearest neighbors, according to some measure of distance. In our implementation, we chose the cosine distance.

We implemented capturing and restoration of a coroutine executing the k-NN algorithm for classifying a relatively small database of 7 MB (but the same procedure can be used for larger databases).

In k-NN, the training table is traversed for every record of the test table, to compute the distance with the training records. We measured the total execution times of the application with and without migration. Without migration, the program simply runs to completion. With migration, it executes in machine A until it reaches half the number of records in the test base, then yields. At this point the platform initiates the migration of the execution to a machine B, where the state file is transmitted back to A:

1. At A, the current execution state is captured, saved to a file and transmitted to B, along with the working files.
2. At B, the file(s) is received and transmitted back to A.
3. At A, the file(s) is received, loaded and the computation is restored.

Finally, the restored coroutine is executed until its end. The transfer involved a buffer of 14582 kB.

We repeated each experiment seven times to guarantee an error of 0.9% with a confidence interval of 95%. Mean time for execution without migration was $446955ms$, while execution time with migration reached $462506ms$. Total migration time (time for capturing, saving, transmitting, loading and restoring the computation) amounted to $15051ms$. The program was stopped after $225945ms$.

5. Final remarks

This paper argued that programming languages should offer mechanisms for fine-grained capture and restoration of the execution state in order to allow the implementation of different policies for migration and persistence. To illustrate this idea, we presented an API for reifying and installing computations. Our proposal is different from others in allowing the reification of the execution state of running computations in the form of fine-grained data structures that can be freely manipulated by the programmer. With this API, it is possible to control factors that are typically predefined in black-box serialization frameworks, such as granularity, the amount of execution state to be transferred or persisted, and the way the computation will be rebound to the new local context.

We have shown that this approach allows implementing various and powerful functionalities. A drawback is that the programming burden augments, as does the chance of dealing with representation inconsistencies. The idea is that an API such as LuaNua be used in the development of libraries which implement different policies, while still allowing direct access to the API for more specific applications. The extended version of the LOOP library that we described is an example of such a policy-implementing layer.

Future work includes exploring the flexibility we advocated, building libraries with different policies for distributed systems based on Lua. Besides their role in further evaluating our proposal, these will also be used as support in other research, for instance in investigating the management of concurrency levels in concurrent servers and in opportunistic computing.

Referências

[Bouchenak et al. 2004] Bouchenak, S., Hagimont, D., Krakowiak, S., Palma, N. D., and Boyer, F. (2004). Experiences implementing efficient Java thread serialization, mobility and persistence. *Software: Practice & Experience*, 34(4):355–393.

[Clarke 2013] Clarke, I. (2013). Swarm-dpl: A transparently scalable distributed programming language. http://swarmframework.org. Last visited on 13/09/2013.

[Clinger et al. 1999] Clinger, W. D., Hartheimer, A. H., and Ost, E. M. (1999). Implementation strategies for first-class continuations. *Higher Order Symbol. Comput.*, 12:7–45.

[Friedman and Wand 1984] Friedman, D. P. and Wand, M. (1984). Reification: Reflection without metaphysics. In *LFP '84: Proceedings of the 1984 ACM Symposium on LISP and functional programming*, pages 348–355, New York, NY, USA. ACM.

[Germain et al. 2006] Germain, G., Feeley, M., and Monnier, S. (2006). Concurrency oriented programming in Termite Scheme. In *Scheme and Functional Programming Workshop (SFPW'06)*, pages 125–135, New York, NY, USA. ACM.

[Ierusalimschy et al. 2007] Ierusalimschy, R., de Figueiredo, L. H., and Celes, W. (2007). The evolution of Lua. In *HOPL III: Proceedings of the third ACM SIGPLAN conference on History of programming languages*, New York, NY, USA. ACM.

[LuaNua 2013] LuaNua (2013). LuaNua. http://homepages.dcc.ufmg.br/˜anolan/ research/-luanua:start. Last visited on 13/09/2013.

[Maia 2008] Maia, R. (2008). LOOP: Lua Object-Oriented Programming. http://loop.luaforge.net/. Last visited on 13/09/2013.

[Milanés et al. 2008] Milanés, A., Rodriguez, N., and Schulze, B. (2008). State of the art in heterogeneous strong computation migration. *Concurrency and Computation: Practice & Experience*, 20:1485 – 1508.

[Moura and Ierusalimschy 2009] Moura, A. L. D. and Ierusalimschy, R. (2009). Revisiting coroutines. *ACM Transactions on Programming Languages and Systems*, 31:1–31.

[Rivard 1996] Rivard (1996). Smalltalk: a Reflective Language. *In Proceedings of Reflection'96*.

[Rompf et al. 2009] Rompf, T., Maier, I., and Odersky, M. (2009). Implementing first-class polymorphic delimited continuations by a type-directed selective cps-transform. *SIGPLAN Notices*, 44:317–328.

[Stackless 2011] Stackless (2011). Stackless Python. http://www.disinterest.org/resource/ stackless/2.6-docs-html/library/stackless/pickling.html. Last visited on 13/09/2013.

[Sunshine-Hill 2008] Sunshine-Hill, B. (2008). *Lua Programming Gems*, chapter Serialization with Pluto. Lua.org, Rio de Janeiro.

[Ururahy et al. 2002] Ururahy, C., Rodriguez, N., and Ierusalimschy, R. (2002). ALua: Flexibility for Parallel Programming. *Computer Languages, Systems & Structures*, 28(2):155 – 180.